

B.TECH 1ST YEAR · 1ST SEMESTER

C Programming

Unit III — Arrays & Strings

Formula Sheet & Exam Revision Document

 1D & 2D Arrays

 Strings & Functions

 Memory Model

 Sorting & Searching

 25+ Flashcards

 Comparison Tables

 Last-Day Revision Weapon | Topper-Level Exam Guide

Contains: Syntax · Diagrams · Code Patterns · Flashcards · Exam Tips



SECTION 1 & 2 — Introduction to Arrays & Declaration



1. WHAT IS AN ARRAY?

An **array** is a collection of *same data type* elements stored in *contiguous memory locations*, accessed using a single variable name and an index.

Why Arrays?

- Store multiple values under one name
- Avoid declaring 100 separate variables
- Enable loops to process data efficiently
- Facilitate sorting & searching

Characteristics

- Homogeneous elements (same type)
- Contiguous memory allocation
- Zero-based indexing
- Fixed size (decided at compile time)
- Random access via index

```
Array Structure | — Homogeneous Elements | —  
int, float, char... | — Contiguous Memory | —  
Elements side-by-side | — Indexed Access | —  
arr[0], arr[1]... | — Fixed Size — Declared at  
compile time
```



2. ARRAY DECLARATION & INITIALIZATION

Syntax

```
data_type array_name[size];
```

```
// Examples:
```

```
int arr[5];  
float marks[10];  
char name[20];
```

Types of Initialization

① Static (Compile-time)

```
int arr[5] = {10, 20, 30, 40, 50};
```

② Partial Initialization

```
int arr[5] = {1, 2};  
// arr[2], arr[3], arr[4] → 0
```

③ Size Omitted

```
int arr[] = {5, 10, 15};  
// Compiler counts → size = 3
```

④ Dynamic (Runtime)

Memory Representation

```
int arr[5] = {10, 20, 30, 40, 50};
```

10	20	30	40	50
arr[0]	arr[1]	arr[2]	arr[3]	arr[4]
1000	1002	1004	1006	1008

✦ Key Rules

✓ Index always starts from
0

✓ Last valid index =
size - 1

✓ Default value =
0

(global/static)
✓ `int` takes
2 or 4 bytes

✓ Addresses are consecutive

```
int arr[5], i;  
for(i=0; i<5; i++)  
    scanf("%d", &arr[i]);
```

⚠ Common Mistake

`arr[5]` in a size-5 array is OUT OF BOUNDS!

Valid range: `arr[0]` to `arr[4]`

SECTION 3 & 4 — Indexing, Memory Model & Array Operations

3. ARRAY INDEXING & MEMORY MODEL

ADDRESS CALCULATION FORMULA

$\text{Address}(\text{arr}[i]) = \text{Base} + (i \times \text{sizeof}(\text{type}))$

Example

Given: `int arr[5]`, Base = 2000, `sizeof(int)` = 2

Element	Index (i)	Address = 2000 + i×2
arr[0]	0	2000
arr[1]	1	2002
arr[2]	2	2004
arr[3]	3	2006
arr[4]	4	2008

Memory Layout Diagram

RAM Layout:

```
10 | @ 2000 | | arr[0] =  
= 20 | @ 2002 | | arr[1]  
arr[2] = 30 | @ 2004 | |  
| arr[3] = 40 | @ 2006 |  
| | arr[4] = 50 | @ 2008  
| | Each int = 2 bytes →  
addresses differ by 2
```

★ Key Points

✓ Array name =
pointer to first element

✓ `arr == &arr[0]`
✓ `*(arr+i) == arr[i]`

4. OPERATIONS ON ARRAYS

Traversal

```
for(i=0; i<n; i++)  
    printf("%d ", arr[i]);
```

Insertion at End

```
// Increase size, add at n  
arr[n] = value;  
n++;
```

Deletion at Position

```
for(i=pos; i<n-1; i++)  
    arr[i] = arr[i+1];  
n--;
```

Linear Search

```
for(i=0; i<n; i++) {  
    if(arr[i] == key) {  
        printf("Found at %d", i);  
        break;  
    }  
}
```

Linear Search

O(n) time · Works on unsorted arrays · Simple loop

Bubble Sort (Basic)

```
for(i=0; i<n-1; i++)  
    for(j=0; j<n-i-1; j++)  
        if(arr[j]>arr[j+1]){  
            temp=arr[j];  
            arr[j]=arr[j+1];  
            arr[j+1]=temp;  
        }
```

Bubble Sort

O(n²) · Swap adjacent pairs · n-1 passes

+ Sum & Average

```
int arr[5]={10,20,30,40,50};
int sum=0, i;
float avg;

for(i=0; i<5; i++)
    sum += arr[i];

avg = (float)sum/5;
printf("Sum=%d Avg=%.2f",
       sum, avg);
```

Logic:

Accumulate sum in loop, divide by n for average.

↺ Reverse Array

```
// Method 1: Two-pointer
int start=0, end=n-1;
while(start < end){
    temp = arr[start];
    arr[start] = arr[end];
    arr[end] = temp;
    start++;
    end--;
}

// Method 2: Print reverse
for(i=n-1; i>=0; i--)
    printf("%d ", arr[i]);
```

Logic:

Swap from both ends moving inward. n/2 swaps total.

💡 Problem Patterns

Pattern: Count Occurrences

```
int count=0;
for(i=0; i<n; i++){
    if(arr[i]==key)
        count++;
}
```

Pattern: Second Maximum

```
max1=max2=arr[0];
for(i=1; i<n; i++){
    if(arr[i]>max1){
        max2=max1;
        max1=arr[i];
    }else if(arr[i]>max2)
        max2=arr[i];
}
```

Pattern: Positive/Negative Count

```
int pos=0, neg=0;
for(i=0; i<n; i++){
    if(arr[i]>0) pos++;
    else if(arr[i]<0) neg++;
}
```

🔍 Max & Min

```
int max=arr[0], min=arr[0];

for(i=1; i<n; i++){
    if(arr[i] > max)
        max = arr[i];
    if(arr[i] < min)
        min = arr[i];
}

printf("Max=%d Min=%d",
       max, min);
```

Logic:

Initialize with arr[0]; compare each element.

🔍 Search Element

```
int found=0;
for(i=0; i<n; i++){
    if(arr[i]==key){
        printf("At index %d",i);
        found=1;
        break;
    }
}

if(!found)
    printf("Not found");
```

Key:

Use flag variable. Break on first match.



SECTION 6 — Two-Dimensional Arrays



6. TWO-DIMENSIONAL ARRAYS

Definition & Syntax

A 2D array stores data in **rows and columns** (matrix form).

DECLARATION SYNTAX

data_type arr[rows][cols];

```
int mat[3][3];
int mat[3][3] = {
    {1,2,3},
    {4,5,6},
    {7,8,9}
};
```

Matrix Layout

a[0] [0]	a[0] [1]	a[0] [2]
a[1] [0]	a[1] [1]	a[1] [2]
a[2] [0]	a[2] [1]	a[2] [2]

ADDRESS FORMULA (2D)

Base + (i×C + j) × sizeof(type)

C = number of columns

Input / Output

```
// Input 2D array
for(i=0; i<r; i++)
    for(j=0; j<c; j++)
        scanf("%d", &mat[i][j]);

// Display matrix
for(i=0; i<r; i++){
    for(j=0; j<c; j++)
        printf("%4d", mat[i][j]);
    printf("\n");
}
```

Matrix Addition

```
for(i=0; i<r; i++)
    for(j=0; j<c; j++)
        C[i][j]=A[i][j]+B[i][j];
```

Matrix Transpose

```
for(i=0; i<r; i++)
    for(j=0; j<c; j++)
        T[j][i] = A[i][j];
// Swap rows ↔ columns
```

Matrix Multiplication

```
for(i=0; i<r1; i++)
    for(j=0; j<c2; j++){
        C[i][j]=0;
        for(k=0; k<c1; k++)
            C[i][j]+=A[i][k]*B[k][j];
    }
```

Condition:

Columns of A must equal Rows of B

🚀 1D vs 2D Array Comparison

Feature	1D Array	2D Array
Declaration	<code>int a[5]</code>	<code>int a[3][3]</code>
Indexing	<code>a[i]</code>	<code>a[i][j]</code>

Loops needed	1 loop	2 nested loops
Use case	List / Vector	Matrix / Table
Total elements	n	$r \times c$

7. INTRODUCTION TO STRINGS

A **string** is a *one-dimensional array of characters* terminated by the null character `'\0'` (ASCII 0).

STRING DECLARATION

```
char str[size]; // or char str[] =
"Hello";
```

```
char str1[10];
char str2[10] = {'H','i','\0'};
char str3[] = "Hello";
char str4[10] = "World";
```

Memory Representation of "Hello"

H	e	l	l	o	\0
[0]	[1]	[2]	[3]	[4]	[5]

⚠️ NULL Terminator Rule

Always declare `size + 1` to accommodate `'\0'`.
 "Hello" needs `char str[6]` — 5 chars + 1 for `'\0'`

Key Rules

- Strings end with `'\0'` (null terminator)
- Use `#include <string.h>` for string functions
- "Hello" automatically adds `'\0'`
- String length = characters before `'\0'`
- Strings can't be assigned with `=` after declaration (use `strcpy`)

📌 Important Note

```
sizeof("Hello") = 6 (includes '\0')
strlen("Hello") = 5 (excludes '\0')
sizeof(str) ≠ strlen(str)
```

String vs Character Array

String	Character Array
Ends with <code>'\0'</code>	May not have <code>'\0'</code>
"Hello"	<code>{'H','e','l','l','o'}</code>
Use string functions	Can't use directly

8. STRING INPUT / OUTPUT

Input Functions

```
// scanf - stops at space
scanf("%s", str);

// gets - reads full line (unsafe)
gets(str);

// fgets - safe, reads n-1 chars
fgets(str, 50, stdin);

// Output
printf("%s", str); // no newline
puts(str);         // adds newline
```

Comparison Table

Function	Safe?	Stops At	Use
<code>scanf()</code>	✓	Space/newline	Single word
<code>gets()</code>	✗ Unsafe	Newline	Avoid in production
<code>fgets()</code>	✓ Safe	Newline or n-1	Recommended
<code>puts()</code>	Output	—	Print + newline

⚠️ Never Use gets()

`gets()` causes buffer overflow — use `fgets()` instead!

9. STRING FUNCTIONS

Function	Purpose	Syntax	Example / Returns
<code>strlen()</code>	Length of string	<code>strlen(str)</code>	<code>strlen("Hello")</code> → 5
<code>strcpy()</code>	Copy string	<code>strcpy(dest, src)</code>	Copies src into dest
<code>strcat()</code>	Concatenate	<code>strcat(dest, src)</code>	Appends src to dest
<code>strcmp()</code>	Compare strings	<code>strcmp(s1, s2)</code>	0=equal, <0, >0
<code>strupr()</code>	Convert uppercase	<code>strupr(str)</code>	"hello" → "HELLO"
<code>strlwr()</code>	Convert lowercase	<code>strlwr(str)</code>	"HELLO" → "hello"
<code>strrev()</code>	Reverse string	<code>strrev(str)</code>	"Hello" → "olleH"
<code>strncpy()</code>	Copy n chars	<code>strncpy(d, s, n)</code>	Safe version of strcpy

strcmp() Return Values

```
strcmp(s1, s2) :
```

- Returns

0

→ strings are equal

- Returns

< 0

→ s1 < s2 (alphabetically)

- Returns

> 0

→ s1 > s2 (alphabetically)

```
if(strcmp(s1, s2) == 0)
    printf("Equal");
else
    printf("Not Equal");
```

strlen() vs sizeof()

Property	strlen()	sizeof()
Counts	Chars till '\0'	Total memory
Includes '\0'	No	Yes
Header	string.h	Built-in
"Hello"	5	6
char s[20]="Hi"	2	20



10. STRING PROGRAMS

String Length (Manual)

```
int len=0;
while(str[len] != '\0')
    len++;
printf("Length = %d", len);
```

Reverse String (Manual)

```
int n=strlen(str);
char rev[50];
int j=0;
for(i=n-1; i>=0; i--)
    rev[j++] = str[i];
```

String Comparison

```
gets(s1); gets(s2);
if(strcmp(s1,s2)==0)
    printf("Equal");
```

Count Vowels

```
int v=0, c=0, i=0;
while(str[i] != '\0'){
    char ch=tolower(str[i]);
    if(ch=='a' || ch=='e' ||
       ch=='i' || ch=='o' ||
       ch=='u') v++;
    else c++;
    i++;
}
```

```
rev[j] = '\0';
printf("%s", rev);
```

```
else
    printf("Not Equal");
```

Palindrome Check

```
int n=strlen(str);
int flag=1;
for(i=0; i<n/2; i++)
    if(str[i]!=str[n-1-i])
        flag=0;
if(flag)
    printf("Palindrome");
else
    printf("Not Palindrome");
```

Copy Without strcpy

```
i=0;
while(src[i]!='\0'){
    dest[i]=src[i];
    i++;
}
dest[i]='\0';
```

Key Tips

- ✓ Always terminate with '\0'
- ✓ Use `tolower()` for case-insensitive
- ✓ `isalpha()` , `isdigit()` in `ctype.h`

SECTION 11 — Important Comparison Tables



Array vs String

Feature	Array	String
Type	Any data type	char only
Terminator	None required	'\0' required
Example	<code>int a[5]</code>	<code>char s[10]</code>
Input	scanf with loop	scanf("%s")
Functions	Manual logic	string.h
Print	Loop printf	printf("%s")
Compare	Element-wise	strcmp()



1D vs 2D Arrays

Feature	1D Array	2D Array
Dimensions	One	Two (row, col)
Declaration	<code>int a[n]</code>	<code>int a[r][c]</code>
Access	<code>a[i]</code>	<code>a[i][j]</code>
Loops	Single loop	Nested loop
Use case	List, vector	Matrix, table
Memory	Linear	Row-major order
Size	n elements	r×c elements



strcpy() vs strcat()

Feature	strcpy()	strcat()
Action	Copies string	Appends string
Overwrites	Yes	No (appends)
Syntax	<code>strcpy(d, s)</code>	<code>strcat(d, s)</code>
Result	d = s	d = d + s
Old dest data	Lost	Preserved



strlen() vs sizeof()

Feature	strlen()	sizeof()
Purpose	String length	Memory size
Counts '\0'	No	Yes
Header	string.h	Built-in
char s[20]="Hi"	2	20
"Hello"	5	6
Type	Function	Operator



gets() vs fgets()

Feature	gets()	fgets()
Safety	 Unsafe	 Safe
Overflow	Possible	Prevented
Size limit	None	Specified by user
Newline	Removes '\n'	Keeps '\n'
Syntax	<code>gets(str)</code>	<code>fgets(s, n, stdin)</code>



Linear Search vs Bubble Sort

Feature	Linear Search	Bubble Sort
Purpose	Find element	Sort array
Time	O(n)	O(n ²)
Best case	O(1)	O(n)
Sorted needed?	No	No (but sorts)

Recommended

☐ No

☒ Yes

Swaps

None

Many

📖 12. KEY TERMS & DEFINITIONS (35 Terms)

Array → Collection of same-type elements in contiguous memory

Element → Individual value in an array

Subscript → Another name for index

Initialization → Assigning values at declaration

Linear Search → Search one-by-one from start to end

2D Array → Array with rows and columns (matrix)

Transpose → Swap rows and columns of matrix

String → Character array ending with null '\0'

strlen() → Returns length excluding '\0'

strcat() → Appends src to end of dest

gets() → Reads full line (unsafe — avoid)

puts() → Prints string and adds newline

Buffer Overflow → Writing beyond array bounds

Vowel → a, e, i, o, u (and uppercase)

tolower() → Converts char to lowercase (ctype.h)

isalpha() → Checks if char is alphabet

strrev() → Reverses string in place

Partial Init → Initialize few elements; rest become 0

Index → Position of element; starts at 0

Base Address → Memory address of arr[0]

Contiguous → Elements stored side-by-side in RAM

Traversal → Visiting every element of an array

Bubble Sort → Sort by comparing adjacent pairs repeatedly

Matrix → Rectangular arrangement of data (rows × cols)

Row-Major → 2D arrays stored row-by-row in memory

Null Character → '\0' (ASCII 0) marks end of string

strcpy() → Copies src string to dest

strcmp() → Compares two strings lexicographically

fgets() → Safe input with size limit

scanf("%s") → Reads string until whitespace

Palindrome → String same forwards and backwards

Consonant → Alphabetic chars that are not vowels

toupper() → Converts char to uppercase (ctype.h)

isdigit() → Checks if char is digit

sizeof() → Returns total bytes of variable/type

Out-of-Bounds → Accessing index beyond valid range

📐 13. FORMULA & SYNTAX BOX

ARRAY DECLARATION

```
type name[size];
```

1D ADDRESS FORMULA

```
Base + (i × size_of_type)
```

2D ADDRESS FORMULA

```
Base + (i×C + j) ×
size_of_type
```

STRING DECLARATION

```
char str[n+1] = "text";
```

STRING COPY

```
strcpy(destination,
source);
```

STRING CONCAT

```
strcat(destination,
source);
```

2D ARRAY SYNTAX

```
type arr[rows][cols];
```

SAFE STRING INPUT

```
fgets(str, size, stdin);
```

STRING COMPARE

```
strcmp(s1, s2) → 0 / <0 /
>0
```

⚡ 14. COMMON PATTERNS FOR EXAM

Pattern 1: Traverse & Accumulate

```
// Use: sum, count, product
int result = 0;
for(i=0; i<n; i++)
    result += arr[i];
```

Pattern 2: Find Max/Min

```
int max = arr[0];
for(i=1; i<n; i++)
    if(arr[i] > max)
        max = arr[i];
```

Pattern 3: Two-Pointer

```
// Use: reverse, palindrome
int lo=0, hi=n-1;
while(lo < hi){
    // swap arr[lo], arr[hi]
    lo++; hi--;
}
```

Pattern 4: Nested Loop (Matrix)

```
for(i=0; i<r; i++)
    for(j=0; j<c; j++)
        // process mat[i][j]
        printf("%d ", mat[i][j]);
```

Pattern 5: String Traversal

```
// With index
for(i=0; str[i]!='\0'; i++)
    printf("%c", str[i]);

// With pointer style
while(str[i] != '\0')
    process(str[i++]);
```

Pattern 6: Flag Variable

```
// Use: search, palindrome, prime
int flag = 1; // assume true
for(...){
    if(condition_fails)
        flag = 0;
}
if(flag) printf("Yes");
```

Pattern 7: Bubble Sort

```
for(i=0; i<n-1; i++)
    for(j=0; j<n-i-1; j++)
        if(arr[j]>arr[j+1]){
            t=arr[j];
            arr[j]=arr[j+1];
            arr[j+1]=t;
        }
```

Pattern 8: Diagonal Sum

```
// Main diagonal (i==j)
int sum=0;
for(i=0; i<n; i++)
    sum += mat[i][i];
```

Pattern 9: Upper Triangle

```
for(i=0; i<n; i++)
    for(j=i+1; j<n; j++)
        printf("%d ", mat[i][j]);
```

💡 Diagonal Check

Main Diag: $i == j$

Anti-Diag: $i + j == n-1$



SECTION 15 — Flashcards (35 Q&A)

Q: What is an array?

A: Collection of similar data type elements stored in contiguous memory locations.

Q: What is the address formula for arr[i]?

A: Base Address + (i × sizeof(data type)).

Q: What is a string in C?

A: A character array terminated by the null character '\0'.

Q: What does sizeof("Hello") return?

A: 6 (includes the null terminator '\0').

Q: Why is gets() unsafe?

A: No size limit — causes buffer overflow. Use fgets() instead.

Q: What does strcat(d, s) do?

A: Appends string s to the end of string d.

Q: How to declare a 2D array?

A: int arr[rows][cols]; e.g., int mat[3][3];

Q: What header is needed for string functions?

A: #include <string.h>

Q: What is partial initialization?

A: Initializing fewer elements than size; remaining become 0.

Q: How many loops for 2D array traversal?

A: Two nested loops — one for row (i), one for column (j).

Q: What is Linear Search time complexity?

A: O(n) — scans each element one by one.

Q: What does puts(str) do?

A: Prints string and automatically adds a newline character.

Q: What condition in matrix multiplication?

A: Columns of A must equal Rows of B (c1 == r2).

Q: Main diagonal condition in matrix?

A: i == j (row index equals column index).

Q: What is the index of first element?

A: 0 (zero-based indexing in C).

Q: What is '\0'?

A: Null character (ASCII 0) marking the end of a string.

Q: What does strlen("Hello") return?

A: 5 (counts characters before '\0', excluding it).

Q: What does strcmp(s1,s2) return if equal?

A: 0.

Q: What does strcpy(d, s) do?

A: Copies string s into d, overwriting existing content of d.

Q: Difference between strcpy and strcat?

A: strcpy replaces dest; strcat appends to dest.

Q: What is Bubble Sort's time complexity?

A: O(n²) — two nested loops with n-1 passes.

Q: How to read a string with spaces?

A: Use fgets(str, size, stdin) — not scanf("%s").

Q: What is out-of-bounds access?

A: Accessing arr[n] when valid range is arr[0] to arr[n-1].

Q: What is matrix transpose?

A: Swapping rows and columns: T[j][i] = A[i][j].

Q: How to check palindrome in a string?

A: Compare str[i] with str[n-1-i] for i from 0 to n/2.

Q: What is the 2D address formula?

A: Base + (i×Cols + j) × sizeof(type).

Q: How to reverse array without extra space?

A: Two-pointer: swap arr[lo] and arr[hi] while lo < hi.

Q: What header for tolower(), isalpha()?

A: #include <ctype.h>

Q: How does fgets() differ from gets()?

A: fgets() limits input size and prevents buffer overflow.

Q: char s[20]="Hello" — strlen vs sizeof?

A: strlen = 5; sizeof = 20.

Q: strrev("abcd") returns?

A: "dcba" — reverses the string in place.

Q: Can we assign strings with = after declaration?

A: No — use strcpy(dest, src) instead.

Q: What is row-major order?

A: 2D arrays stored row by row in contiguous memory.

Q: How to find sum of main diagonal?

A: for(i=0;i<n;i++) sum += mat[i][i];

Q: What is the last valid index of int arr[10]?

A: arr[9] — indices are 0 to 9.

❌ COMMON MISTAKES TO AVOID

❌ Mistake 1: Index Out of Bounds

`int arr[5]; arr[5] = 10;` ← WRONG
Valid: `arr[0]` to `arr[4]` only!

❌ Mistake 2: Forgetting '\0' in Strings

`char s[5] = "Hello";` ← OVERFLOW
Correct: `char s[6] = "Hello";` (5 + 1)

❌ Mistake 3: Using `gets()` Unsafely

`gets(str);` ← Buffer overflow risk
Use: `fgets(str, 50, stdin);`

❌ Mistake 4: Wrong Loop Limits

`for(i=0; i<=n; i++)` ← Accesses `arr[n]`
Correct: `for(i=0; i<n; i++)`

❌ Mistake 5: Assigning Strings with `=`

`str = "Hello";` ← WRONG after declaration
Use: `strcpy(str, "Hello");`

❌ Mistake 6: Comparing Strings with `==`

`if(s1 == s2)` ← Compares addresses!
Use: `if(strcmp(s1, s2) == 0)`

❌ Mistake 7: Missing `string.h` Header

Always add `#include <string.h>` for string functions

✅ WRITING TIPS FOR EXAMS

✅ Tip 1: Start Programs with Proper Headers

Always write `#include <stdio.h>` and `int main()` with `return 0`.

✅ Tip 2: Draw Memory Diagrams

Draw array boxes with index numbers below — gives extra marks!

✅ Tip 3: Show Dry Run / Trace

For loops — show a small iteration trace table (i, `arr[i]`, output).

✅ Tip 4: Write Syntax Correctly

`strcpy(dest, src)` — dest first, src second. Same for `strcat`.

✅ Tip 5: Mention Time Complexity

Linear Search: $O(n)$ · Bubble Sort: $O(n^2)$ — mention for extra marks.

✅ Tip 6: Always Use Comments in Code

Add `// Initialize max` type comments to show understanding.

✅ Tip 7: State Conditions Clearly

For matrix multiplication, state: "c1 must equal r2".

🎯 MARKS-FETCHING POINTS

High-Value Topics to Prepare Well

- ✓ Address formula (very common 2-marker)
- ✓ String function syntax table
- ✓ Matrix multiplication program
- ✓ Palindrome program
- ✓ Bubble sort step-by-step
- ✓ 1D vs 2D, `strlen` vs `sizeof` comparisons
- ✓ `gets()` vs `fgets()` — safety explanation



SECTION 17 — QUICK REVISION SHEET (Last-Minute Cheat Sheet)

ARRAY SYNTAX

- `int arr[5];`
- `int arr[5] = {1,2,3,4,5};`
- `int arr[] = {1,2,3};` → size auto
- Index: 0 to n-1
- Last valid: `arr[n-1]`

KEY FORMULAS

- 1D: `Base + i × sizeof(type)`
- 2D: `Base + (i×C + j) × sizeof(type)`
- `strlen("Hi") = 2`, `sizeof("Hi") = 3`
- Diagonal: `i == j`
- Reverse loop: `for(i=n-1;i>=0;i--)`

STRING SYNTAX

- `char str[10];`
- `char str[] = "Hello";`
- Always needs `'\0'` at end
- Safe input: `fgets(s,n,stdin)`
- Output: `puts(str)` or `printf("%s",str)`

STRING FUNCTIONS

- `strlen(s)` → length (no `'\0'`)
- `strcpy(d,s)` → copy s to d
- `strcat(d,s)` → append s to d
- `strcmp(s1,s2)` → 0/neg/pos
- `strrev(s)`, `strupr(s)`, `strlwr(s)`

IMPORTANT RULES

- Never `s1 == s2` → use `strcmp`
- Never `s = "text"` → use `strcpy`
- Never `gets()` → use `fgets()`
- 2D loop always nested (i, j)
- Matrix mult: `cols_A == rows_B`

KEY DIFFERENCES

- `strlen` ≠ `sizeof`
- `strcpy` (replace) ≠ `strcat` (append)
- `gets` (unsafe) ≠ `fgets` (safe)
- 1D (1 loop) ≠ 2D (2 loops)
- Array (any type) ≠ String (char only)

ALGORITHM STEPS

- Max:** init `max=a[0]`, compare each
- Reverse:** swap lo/hi inward
- Sort:** n-1 passes, n-i-1 compare
- Palindrome:** compare `str[i]` & `str[n-1-i]`
- Search:** loop, compare, break

EXAM ALERTS

- ⚠ `arr[n]` is out of bounds!
- ⚠ "Hello" needs `char[6]`, not `char[5]`
- ⚠ `strcmp()` needs `<string.h>`
- ⚠ `toupper/tolower` needs `<ctype.h>`
- ⚠ Matrix: always initialize `C[i][j]=0`



COMPLETE FUNCTION REFERENCE

Function	Header	Syntax	Returns / Effect	Example
<code>strlen()</code>	<code>string.h</code>	<code>strlen(str)</code>	int — length (no <code>'\0'</code>)	<code>strlen("Hi") → 2</code>
<code>strcpy()</code>	<code>string.h</code>	<code>strcpy(d,s)</code>	char* — copies s to d	<code>strcpy(a,"Hi")</code>
<code>strcat()</code>	<code>string.h</code>	<code>strcat(d,s)</code>	char* — appends s to d	<code>strcat(a," World")</code>
<code>strcmp()</code>	<code>string.h</code>	<code>strcmp(s1,s2)</code>	0 / neg / pos	<code>strcmp("ab","ab") → 0</code>
<code>strrev()</code>	<code>string.h</code>	<code>strrev(str)</code>	Reversed string	<code>strrev("abc") → "cba"</code>
<code>strupr()</code>	<code>string.h</code>	<code>strupr(str)</code>	Uppercase string	<code>strupr("hi") → "HI"</code>
<code>strlwr()</code>	<code>string.h</code>	<code>strlwr(str)</code>	Lowercase string	<code>strlwr("HI") → "hi"</code>
<code>fgets()</code>	<code>stdio.h</code>	<code>fgets(s,n,stdin)</code>	Safe line input	<code>fgets(str,50,stdin)</code>

<code>puts()</code>	<code>stdio.h</code>	<code>puts(str)</code>	Print + newline	<code>puts("Hello")</code>
<code>tolower()</code>	<code>ctype.h</code>	<code>tolower(ch)</code>	Lowercase char	<code>tolower('A') → 'a'</code>
<code>toupper()</code>	<code>ctype.h</code>	<code>toupper(ch)</code>	Uppercase char	<code>toupper('a') → 'A'</code>
<code>isalpha()</code>	<code>ctype.h</code>	<code>isalpha(ch)</code>	Non-zero if alpha	<code>isalpha('a') → true</code>

🎯 All the best for your exam! You've got this! 💪

C Programming · Unit III: Arrays & Strings · B.Tech 1st Year